

SuperAgents OS v3.7

Комплексный аудит кода, логики и архитектуры

Репозиторий: github.com/n95887174-source/ai-os-new

Дата: 10 мая 2026 г.

Подготовлено: Z.ai Automated Code Auditor

Всего файлов	Критических	Высоких	Средних	Общая оценка
~80	16	27	51	5/10

Содержание

1. Введение и методология	4
2. Сводная таблица оценок готовности	4
3. Ядро (Core) — оценка: 5.5/10	6
3.1. Kernel.ts	6
3.2. EventBus (events.ts)	6
3.3. DatabaseService.ts	7
3.4. PluginSDK.ts	7
3.5. IntelligenceDSL.ts / Bootstrap.ts	7
3.6. SecurityService.ts	8
3.7. ProviderTracker.ts / WeightOptimizer.ts	8
3.8. Рекомендации для Core (до 9/10)	8
4. Сервисный слой (Services) — оценка: 5.5/10	9
4.1. Критические проблемы безопасности	9
4.2. Критические логические ошибки	9
4.3. Системные проблемы	10
4.4. Рекомендации для Services (до 9/10)	10
5. Провайдеры, Типы и Stores — оценка: 5.5/10	11
5.1. Критические проблемы безопасности	11
5.2. Критические логические ошибки	11
5.3. Архитектурные проблемы	11

5.4. Рекомендации (до 9/10)	12
6. Панели пользовательского интерфейса	12
6.1. Критические UI-проблемы	12
6.2. Проблемы производительности	13
6.3. Проблемы доступности и UX	13
6.4. Рекомендации для UI (до 9/10)	13
7. Системные архитектурные проблемы	14
7.1. Отсутствие валидации данных	14
7.2. Циклические мутации	14
7.3. Отсутствие модели безопасности плагинов	14
7.4. Нет маршрутизации	14
7.5. Отсутствие тестового покрытия	15
8. Приоритетный план доведения до 9/10	15
8.1. Фаза 1 — Критические исправления (1-2 недели)	15
8.2. Фаза 2 — Архитектурные улучшения (2-4 недели)	15
8.3. Фаза 3 — Полировка и продакшн (4-8 недель)	16
9. Заключение	17

1. Введение и методология

Данный отчёт представляет собой результат комплексного аудита проекта SuperAgents OS (v3.7) — событийно-ориентированной среды выполнения для оркестрации AI-агентов. Проект реализован на React 19 + TypeScript + Vite и использует IndexedDB (Dexie), Web Workers, React Flow для визуального построения DAG и Transformers.js для семантического поиска.

Аудит охватывает все уровни архитектуры: ядро (Core), сервисный слой (Services), слой провайдеров (Providers), хранилища состояния (Stores), типы (Types) и все 22+ панели пользовательского интерфейса. Для каждого модуля проведён анализ ошибок кода, логических дефектов, архитектурных проблем, уязвимостей безопасности и производительности.

Каждый модуль получил оценку готовности по шкале от 1 до 10, а также получил конкретные рекомендации по доведению до уровня 9/10. Общая готовность проекта составляет **5/10** — функциональный каркас работает, но содержит множество критических и высоких дефектов, которые требуют приоритетного исправления.

Методология оценки:

1 = Неработающий код / заглушки; 3 = Базовая функциональность с критическими ошибками; 5 = Работающий прототип с значительными дефектами; 7 = Стабильная функциональность с незначительными проблемами; 9 = Продакшн-готовность с минимальными улучшениями; 10 = Безупречная реализация.

2. Сводная таблица оценок готовности

Модуль	Файлы	Оценка
Ядро (Core)	Kernel, EventBus, DatabaseService, PluginSDK, IntelligenceDSL, Bootstrap, SafetyContract, SecurityService, ProviderTracker, WeightOptimizer, storage	5.5/10
Сервисы (Services)	ChatService, OrchestrationService, MemoryService, SandboxService, KeyService, AgentService, RoleService, SkillService, ToolService, TraceService, CognitiveService, RouterService, AdvisorService, DebateService, PolicyService, HealthCheck, MetricsService, SettingsService, SnapshotService, AdminService, MCPService, PricingService	5.5/10
Провайдеры + Типы + Stores	AdapterRegistry, OpenRouterAdapter, GeminiAdapter, OpenAiCompatibleAdapter, useKeyStore, useChatStore, domain.ts, chat.ts, metrics.ts, memory.ts, role.ts	5.5/10
Панель управления (App)	App.tsx, main.tsx	7.5/10

Live Cognition	MissionControl, LiveWorkspace	6.0/10
Chat Panel	ChatPanel	5.0/10
Builder Panel	CognitiveBuilder	6.0/10
Dashboard	DashboardPanel, AgentLiveBoard, LiveEventFeed, RacingWinners, IntelligenceGraph, PredictiveQuota	6.3/10
Provider Manager	ProviderManager, ProviderDetailModal, BrowseModelsView, InstalledProvidersView, RoutingSLAView	6.0/10
Traces Panel	TracesPanel, DecisionGraph, CognitiveMicroscope	6.7/10
Key Table	KeyProfileExtended, OverviewTab, QualityTab, SandboxTab, TracesTab, ToolsTab, NotesTab	7.1/10
Agents Panel	AgentsPanel	6.0/10
Roles Panel	RolesPanel	7.0/10
Skills Panel	SkillsPanel	7.0/10
Tools Panel	ToolsPanel	6.0/10
Connectors Panel	ConnectorsPanel	7.0/10
Memory Panel	MemoryPanel	6.0/10
Knowledge Panel	KnowledgePanel	6.0/10
Health Panel	HealthPanel	7.0/10
Settings Panel	SettingsPanel	8.0/10
Events Panel	EventsPanel	7.0/10
Analytics Panel	AnalyticsPanel	6.0/10
Documentation	DocumentationPanel	8.0/10
Tasks Panel	TasksPanel	5.0/10
Debate Panel	DebatePanel	5.0/10
Aquarium Panel	AquariumPanel	4.0/10
Hive Panel	HivePanel	4.0/10
Chat Admin	ChatAdminPanel	5.0/10
Model Browser	ModelBrowser	7.0/10
Add Key Modal	AddKeyModal	5.0/10
Common	ErrorBoundary, VaultLock	6.0/10
CSS (index.css)	Глобальные стили, переменные, анимации	5.0/10

3. Ядро (Core) — оценка: 5.5/10

3.1. Kernel.ts

Kernel является центральным хранилищем состояния системы, управляя метриками провайдеров, адаптивными весами и режимами SLA. Обнаружены следующие критические проблемы:

Утечка памяти через setInterval: Метод `setInterval()` вызывается в конструкторе, но `clearInterval()` никогда не вызывается. При горячей перезагрузке модуля (HMR) интервалы накапливаются, что приводит к утечке памяти. Отсутствует метод `destroy()` для очистки.

Деление на ноль в SafetyContract: В `enforceSafetyContract` при нормализации весов используется `1.0/sum`. Если все эффективные веса равны 0 (экстремальный adaptive drift), `sum=0`, что даёт Infinity или NaN, проникающий во все веса системы.

Поверхностное слияние состояния: Оператор `{...initialState, ...savedState}` выполняет поверхностное слияние. Если `savedState.weights` имеет старую структуру, она полностью заменяет новую, теряя добавленные поля. Нет системы миграции версий.

Отсутствие обработки localStorage: Вызовы `localStorage.getItem/setItem` без `try/catch`. В приватном режиме или при превышении квоты будет выброшено исключение, крашащее приложение.

Утечка слушателей событий: Четыре слушателя `eventBus.on()` зарегистрированы, но их функции отписки отбрасываются. Слушатели никогда не удаляются.

3.2. EventBus (events.ts)

Типизированная шина событий с 60+ типами событий — центральная нервная система проекта. Основные проблемы:

15+ событий с типом payload any: Типы `trace:updated`, `agent:config_updated`, `system:command`, `cognitive:step:add`, `tool:execution:start`, `debate:updated`, `policy:violation`, `snapshot:captured` и другие используют `any`, полностью разрушая типобезопасность `EventMap`.

Отсутствие метода once(): Нет поддержки одноразовых подписок, необходимых для жизненного цикла (`init complete`, `first connection`).

Нет removeAllListeners() или clear(): Невозможно очистить все подписки, что критично для тестирования и перезапуска модулей.

Дублирующий API EVENTS: Объект EVENTS совместимости дублирует типизированную EventMap, удваивая обслуживание.

3.3. DatabaseService.ts

Сервис базы данных на базе Dexie (IndexedDB) с 9 таблицами и устаревшим SQL-прокси.

Парсинг SQL через includes(): Метод query() использует sql.includes("SELECT") для определения типа запроса. Это крайне хрупко: запрос DELETE FROM notes WHERE id = (SELECT ...) попадёт в ветку SELECT.

Таблица notes — единственная обработанная: Все остальные таблицы (memories, apiKeys, sessions, traces и пр.) возвращают {rows: [], affectedRows: 0} — тихий по-ор для любых запросов к ним.

Позиционный деструктуризатор без валидации: const [id, keyId, text, type, author, timestamp] = params — если вызывающий передаст параметры в неверном порядке, данные будут молча сохранены в неправильные поля.

3.4. PluginSDK.ts

Уязвимость инъекции событий: eventBus.emit(event as keyof EventMap, data) позволяет плагину отправить любое событие, включая kernel:updated с поддельным состоянием или key:removed для удаления ключей. Нет системы разрешений.

Потеря данных в storage.set: String(value) конвертирует все значения хранилища в строки. Объекты становятся "[object Object]", массивы — разделёнными запятыми строками. Необходимо JSON.stringify().

Нет метода unregister(): Плагины загружаются, но никогда не выгружаются корректно. onUnload определён в интерфейсе, но никогда не вызывается.

3.5. IntelligenceDSL.ts / Bootstrap.ts

DSL топологий определяет когнитивные процессы как DAG с типизированными узлами, рёбрами и политиками. Однако политики (latency, privacy, cost, safety) **чисто декларативны** — нет механизма их исполнения во время выполнения. Условие на рёбрах (condition?: string) — заглушка без вычислителя. Топологии не валидируются на предмет висячих рёбер, циклов или отсутствующих entry-точек.

Bootstrap всегда монтирует AuditorTopology без возможности конфигурации. Если orchestrator.mount() выбрасывает исключение, isLocked уже true, и повторный init() становится по-ор — система застревает в ошибочном состоянии. Нет метода

shutdown()).

3.6. SecurityService.ts

Salt в localStorage: Если атакующий получает доступ к localStorage (XSS), он имеет соль, значительно снижая сложность перебора. Соль не должна храниться рядом с зашифрованными данными.

Утечка информации в сообщениях об ошибках: Сообщение "Incorrect password?" раскрывает ожидаемый режим отказа.

Нет ротации ключей и нет пути обновления итераций PBKDF2: Один мастер-ключ для всего шифрования. 600K итераций достаточно сейчас, но потребуется увеличение в будущем без стратегии миграции.

3.7. ProviderTracker.ts / WeightOptimizer.ts

Отрицательный TPS: $genTime = (latency - ttft) / 1000$ может быть отрицательным, если $ttft > latency$ (возможно при стриминге с разной методикой измерения). Это даёт отрицательный TPS без валидации.

Неверная оценка стоимости: Расчёт использует только input-цену для всех токенов (input + output), занижая стоимость для моделей с дорогим выводом (GPT-4o: input \$2.50, output \$10.00).

Деление на ноль в selectionRate: calculateSelectionRates делит на $total = decisions.length$. Если массив пуст, все selectionRate = NaN.

TPS adaptive delta никогда не обновляется: В WeightOptimizer только ttft и reliability получают адаптивные дельты. TPS-измерение остаётся статичным навсегда.

3.8. Рекомендации для Core (до 9/10)

1. Добавить guard $sum === 0$ в SafetyContract.enforceSafetyContract()
2. Исправить расчёт стоимости — разделить input/output токены с разными ценами
3. Заменить String(value) на JSON.stringify(value) в PluginSDK.storage.set
4. Добавить систему разрешений плагинов (whitelist событий)
5. Добавить валидацию топологий (циклы, висячие рёбра, entry-точки)
6. Добавить destroy()/dispose() ко всем singleton-сервисам
7. Устранить все any-типы в EventMap (заменить на конкретные интерфейсы)
8. Добавить миграцию версий состояния в Kernel (version field + migration chain)

9. Перенести `salt` из `localStorage` в производное от мастер-пароля
10. Реализовать исполнение политик DSL в `OrchestrationService`

4. Сервисный слой (Services) — оценка: 5.5/10

4.1. Критические проблемы безопасности

SandboxService — инъекция кода: Код выполняется через `new Function()` внутри `Web Worker` с доступом к `os.executeTool`. Злоумышленник может выполнить `os.executeTool("t-code", "while(true){}")` для DoS или эксфильтрации данных. Нет валидации входных данных, нет ограничения прав инструмента, нет `rate limit`.

ToolService — доступ плагинов к localStorage: `storage.get/set` даёт плагинам неограниченный доступ к `localStorage`, включая чтение API-ключей. `emit: (event, data) => eventBus.emit(event as never, data as never)` обходит проверку типов, позволяя плагину отправить любое событие.

MCPService — SSRF: `fetch(`${serverUrl}/resource?uri=...)` без валидации URI. Злоумышленный MCP-сервер может направить запрос к внутренним ресурсам. Сломанная логика поиска сервера: `s.url && true` всегда `true` — всегда возвращается первый сервер.

AdvisorService — авто-исполнение исправлений: Если `enableAutoFix=true`, предложения `autoExecutable` выполняются автоматически. Злоумышленное или ошибочное LLM-предложение может отключить провайдеров или изменить маршрутизацию.

4.2. Критические логические ошибки

OrchestrationService — нет обнаружения циклов: Если рёбра образуют цикл, `processNode()` рекурсирует бесконечно до переполнения стека. Параллельные ветки выполняются последовательно (`for...of await`), а не конкурентно (`Promise.all`). Blackboard injection через JSON-парсинг `output` — вектор `prototype pollution`.

KeyService — двойной подсчёт стоимости: `updateMetricsFromResponse` использует `(tokens/1000)*0.01`, а `recordUsage` — `pricingService.calculateCost()`. Несогласованная логика ценообразования. Также `handleProviderError` вызывается с `provider name` вместо `keyId` в `HealthCheckService` — ошибка не привязывается ни к какому ключу.

DebateService — расхожимость `convergenceScore`: Формула `avgOverlap*100 + convergenceScore*0.3` позволяет счётчику расти неограниченно и никогда не сходиться правильно. `setInterval` для `debate loop` может накапливать интервалы,

если `executeArgumentRound` длится дольше `roundDelayMs`.

TraceService и CognitiveService — дублирование: Оба сервиса слушают одни и те же события (`cognitive:step:active`, `cognitive:step:completed`), ведут собственные хранилища трассировок и оба отправляют `trace:updated`. Потребители получают конфликтующие данные.

4.3. Системные проблемы

Фрагментированная стратегия персистенции: Разные сервисы используют разные хранилища без атомарности. `MemoryService` — `Dexie`, `AgentService` — `localStorage`, `RoleService` — `Dexie+localStorage`, `ToolService` — `localStorage`, `SettingsService` — `localStorage`. Невозможно создать целостный бэкап или восстановление.

Антипаттерн token-оценки length/4: Минимум 5 сервисов используют `content.length/4` для оценки токенов. Это систематически неточно для не-английского текста, кода и смешанного контента. Необходим общий токенизатор.

Нет teardown/cleanup: Ни один сервис не реализует `destroy()/dispose()`. Слушатели событий, интервалы и `Workers` утекают при HMR или завершении приложения.

Side-effect singleton pattern: Каждый сервис — синглтон, начинающий асинхронную работу в конструкторе. Это делает тестирование невозможным без мокирования и создаёт скрытые зависимости порядка инициализации.

4.4. Рекомендации для Services (до 9/10)

1. Полная переработка безопасности Sandbox: система разрешений для инструментов, валидация входных данных, таймауты
2. Добавить обнаружение циклов в `OrchestrationService`
3. Унифицировать `TraceService` и `CognitiveService` в единый источник истины
4. Исправить `HealthCheckService` — передавать `keyId` вместо `provider name`
5. Стандартизировать персистенцию — перевести все сервисы на `Dexie`
6. Добавить `debounce` для всех операций сохранения
7. Реализовать блокирующее исполнение политик в `PolicyService`
8. Исправить `MCPService.readResource` — логику поиска сервера по URI
9. Заменить `content.length/4` на общий BPE-токенизатор
10. Добавить `destroy()` ко всем сервисам — очистка `listeners`, `intervals`, `workers`

5. Провайдеры, Типы и Stores — оценка: 5.5/10

5.1. Критические проблемы безопасности

Gemini API Key в URL query parameter: Ключ передаётся как `?key=apiKey` в URL. Прокси-логи записывают полный URL с ключом, история браузера может кэшировать его, Referer-заголовки могут утекать. Необходимо перенести ключ в заголовок `x-goog-api-key` или обрабатывать на уровне прокси.

API-ключи хранятся в открытом виде в Dexie: Тип `metrics.ts` содержит `key: string` — сырой API-ключ в виде `plaintext`. Флаг `isEncrypted` опционален и не `enforced` типовой системой.

Утечка текста ошибки в OpenRouter: В не-стриминговом пути `errorText` передаётся без обрезки, в то время как стриминговый путь корректно обрезает `.slice(0, 200)`. Несогласованность.

5.2. Критические логические ошибки

Side effect внутри state updater (useChatStore): `memoryService.store()` вызывается внутри `setSessions()` updater. React state updater должен быть чистой функцией. В Strict Mode React может вызывать updater дважды, создавая дубликаты в памяти.

deleteSession оставляет orphan activeSessionId: При удалении активной сессии `activeSessionId` устанавливается в `"default"`, но сессии с таким ID может не существовать в отфильтрованном списке.

System prompts теряются в Gemini: Системные сообщения маппируются в `user role`. Gemini API поддерживает `systemInstruction` как отдельное поле верхнего уровня — без него системные промпты теряют свою семантическую роль.

5.3. Архитектурные проблемы

DRY нарушение — SSE parser скопирован 3 раза: Идентичный код парсинга SSE-потока присутствует в `OpenRouterAdapter`, `GeminiAdapter` и `OpenAiCompatibleAdapter`. Необходимо вынести в общий утилитный модуль.

KeyExtendedStats — god-interface с 50+ полями: Интерфейс `spanning 76 строк` покрывает производительность, надёжность, стоимость, качество, поведение, прогнозы, трассировки и обучение. Должен быть декомпозирован на под-интерфейсы.

Отсутствие `retry/backoff`: Ни один адаптер не реализует экспоненциальный `backoff` при 429 `rate-limit`. Нет таймаутов по умолчанию — зависшее соединение блокирует навсегда.

Нет типов для `tool/function calling`: Отсутствует роль `tool`, нет `tool_calls` в типе ответа. Это блокирует агентные сценарии использования инструментов.

5.4. Рекомендации (до 9/10)

1. Немедленно перенести Gemini API key из URL в заголовок
2. Вынести `memoryService.store()` из state updater в `useEffect`
3. Исправить `deleteSession` — устанавливать `activeSessionId` на первую оставшуюся сессию
4. Добавить обработку `systemInstruction` в `GeminiAdapter`
5. Вынести SSE parser в общий `src/services/providers/utils/parseSSEStream.ts`
6. Добавить `retry/backoff` middleware для всех адаптеров
7. Декомпозировать `KeyExtendedStats` на фокусированные под-интерфейсы
8. Добавить типы для `tool/function calling` в `providers/types.ts`
9. Добавить шифрование API-ключей по умолчанию (`isEncrypted` enforced)
10. Обрезать `errorText` во всех путях ошибок

6. Панели пользовательского интерфейса

6.1. Критические UI-проблемы

Неработающие кнопки и заглушки (15+ случаев): `RoutingSLAView` — слайдер `latency threshold` и `toggle Automatic Fallback` полностью нефункциональны (`defaultValue` без `onChange`, `toggle` без `state`). `ToolsTab` — кнопка `Reset Statistics` без `onClick`. `TasksPanel` — кнопка `Refresh` без обработчика. `ChatAdminPanel` — кнопка `Export JSON` без `onClick`. `AddKeyModal` — ссылка `Where to find the key` ведёт на #. `AgentPanel` — тогглы `HIL` и `VPC Isolation` захардкожены (`checked={false}` / `checked={true}`, `onChange={() => {}}`). `MemoryPanel` — кнопки `Export Vectors`, `View Embeddings`, `Delete vector` без обработчиков. `ConnectorsPanel` — `Connect/Disconnect` фейковые, кнопка `Generate URL` без обработчика.

Фейковые/захардкоженные данные (10+ случаев): `MemoryPanel` — `Total Vectors` (`memories.length * 142`), `Embedding Dimensions` (1536), `Index Density` (84%),

Semantic Clarity (96%), Avg Retrieval Latency (12ms), Cognitive Fragments (1240) — всё захардкожено. HealthPanel — ALL SYSTEMS OPERATIONAL всегда зелёный. AnalyticsPanel — все данные графиков mock. PredictiveQuota — $\text{tokensPerHour} = \text{usedTokens} / 4$ (выдуманный делитель). EventsPanel — $\text{EPS} = \text{Math.random()} * 5 + 1$ (случайное число).

CSS — **отсутствующие классы:** .spin используется в 5+ компонентах для анимации загрузки, но не определён в index.css — спиннеры молча не работают. .action-btn используется в ModelBrowser, ChatPanel, KnowledgePanel, но не определён. Нет поддержки Firefox scrollbar (только -webkit-). Нет prefers-reduced-motion.

6.2. Проблемы производительности

AquariumPanel и HivePanel — критическая проблема: Animation useEffect зависит от [mousePos, keys, food/keys]. mousePos обновляется при каждом движении мыши, что приводит к уничтожению и пересозданию 50ms интервала на каждое движение мыши. Это создаёт серьёзные задержки и дёрганье анимации. Необходимо использовать useRef для mousePos вместо useState.

Orphaned timeouts: В AquariumPanel и HivePanel timeoutRef.current = setTimeout(...) перезаписывает предыдущий таймаут без clearTimeout — предыдущие таймауты становятся потерянными и не могут быть очищены.

Внешние ресурсы: AquariumPanel загружает текстуры с transparenttextures.com — не работает оффлайн, утечка IP, риск компрометации домена.

6.3. Проблемы доступности и UX

Поиск в App.tsx — полностью нефункциональный input без value/onChange. Статус RUNTIME ONLINE всегда зелёный. ChatPanel — textarea rows={1} без авто-роста. Нет контекстного окна усечения — вся история отправляется в API. Смешанные языки: AquariumPanel — русский ("Аквариум Интеллекта"), ModelBrowser — русский ("Каталог моделей"), остальной UI — английский.

Отсутствуют aria-label на интерактивных элементах (InstalledProvidersView search, App.tsx search input). Нет :focus-visible стилей. Нет prefers-reduced-motion. Клавиатурная навигация не поддерживается.

6.4. Рекомендации для UI (до 9/10)

1. Добавить .spin и .action-btn CSS классы в index.css
2. Исправить dependency arrays в AquariumPanel/HivePanel — useRef для mousePos

3. Заменить все фейковые данные на реальные вызовы сервисов или убрать метрики
4. Реализовать все неработающие кнопки или визуально отключить с пометкой Coming Soon
5. Добавить error boundary для каждой панели
6. Добавить loading states для всех асинхронных операций
7. Реализовать контекстное окно усечения в useChatStore
8. Заменить alert() на toast/notification систему
9. Добавить aria-label и :focus-visible стили
10. Унифицировать язык UI — выбрать один (английский или русский)

7. Системные архитектурные проблемы

7.1. Отсутствие валидации данных

Данные проходят через цепочку событий → Kernel → ProviderTracker → WeightOptimizer → SafetyContract с as-приведениями типов и нулевой runtime-валидацией. Одно malformed событие может разрушить всё состояние системы. Нет schema validation ни на одном уровне. Необходимо внедрить Zod или подобную библиотеку для валидации критических данных на границах модулей.

7.2. Циклические мутации

Kernel.reduce() вызывает updateProviderMetric (мутация состояния), затем enforceSafetyContract (мутация состояния снова), затем отправляет kernel:updated. Нет границ транзакции — если SafetyContract выбрасывает исключение посреди мутации, состояние обновлено частично. Необходим паттерн immutable state с атомарными обновлениями.

7.3. Отсутствие модели безопасности плагинов

PluginSDK даёт плагинам полный доступ к шине событий без песочницы, ограничения прав или валидации выходных данных. Скомпрометированный или ошибочный плагин может: отправить любое событие, разрушить состояние Kernel, внедрить поддельные метрики, читать/писать в localStorage без ограничений. Необходима система capability-based security.

7.4. Нет маршрутизации

Навигация реализована через `useState + switch` вместо `react-router`. Это создаёт проблемы: невозможно поделиться URL конкретной панели, кнопка Назад не работает, не работает `deep linking`. Для 22+ панелей это серьёзный UX-дефект.

7.5. Отсутствие тестового покрытия

В проекте всего 5 тестовых файлов (`events.test.ts`, `DatabaseService.test.ts`, `OrchestrationService.test.ts`, `SandboxService.test.ts`, `MemoryService.test.ts`). Покрытие сервисного слоя минимально. Нет тестов для UI-компонентов. Нет интеграционных тестов. Для продакшн-готовности необходимо покрытие не менее 80%.

8. Приоритетный план доведения до 9/10

8.1. Фаза 1 — Критические исправления (1-2 недели)

Задача	Описание	Приоритет
Безопасность Sandbox	Система разрешений инструментов, валидация кода, таймауты выполнения	Критический
Gemini API Key утечка	Перенести ключ из URL в заголовок <code>x-goog-api-key</code>	Критический
Side effect в <code>setState</code>	Вынести <code>memoryService.store()</code> из <code>updater</code> в <code>useEffect</code>	Критический
Деление на ноль	Guard <code>sum===0</code> в <code>SafetyContract</code> , guard пустого массива в <code>selectionRate</code>	Критический
PluginSDK <code>storage.set</code>	Заменить <code>String(value)</code> на <code>JSON.stringify(value)</code>	Критический
Отсутствующие CSS классы	Добавить <code>.spin</code> и <code>.action-btn</code> в <code>index.css</code>	Высокий
IntelligenceGraph $O(n*m)$	Заменить <code>find()</code> на <code>Map</code> для поиска узлов по ID	Высокий
<code>deleteSession orphan</code>	Устанавливать <code>activeSessionId</code> на первую оставшуюся сессию	Высокий

Таблица 2. Фаза 1 — Критические исправления

8.2. Фаза 2 — Архитектурные улучшения (2-4 недели)

Задача	Описание	Приоритет
Валидация данных (Zod)	Внедрить <code>schema validation</code> на всех границах модулей (<code>events</code> , <code>kernel state</code> , <code>DB read/write</code>)	Высокий

Система разрешений плагинов	Capability-based security: whitelist событий, scoped storage, validated tool output	Высокий
React Router	Заменить useState навигацию на react-router для URL-поддержки	Высокий
Унификация персистенции	Перевести все сервисы с localStorage на Dexie	Высокий
Общий SSE parser	Вынести дублированный код в utils/parseSSEStream.ts	Средний
Retry/backoff middleware	Экспоненциальный backoff для 429, таймауты по умолчанию	Средний
destroy() для всех сервисов	Очистка listeners, intervals, workers при HMR/shutdown	Средний
Унификация трассировки	Объединить TraceService и CognitiveService	Средний

Таблица 3. Фаза 2 — Архитектурные улучшения

8.3. Фаза 3 — Полировка и продакшн (4-8 недель)

Задача	Описание	Приоритет
Тестовое покрытие 80%+	Unit + integration + e2e тесты для всех сервисов и ключевых UI-компонентов	Средний
Реальные данные в UI	Заменить все захардкоженные метрики на вызовы сервисов	Средний
Error boundaries для панелей	Каждая панель обернута в ErrorBoundary с recovery	Средний
Loading states	Skeleton/spinner для всех асинхронных операций	Средний
Контекстное окно чата	Sliding window с подсчётом токенов для длинных диалогов	Средний
Декомпозиция KeyExtendedStats	Разбить 50+ полей на фокусированные под-интерфейсы	Низкий
Доступность (a11y)	aria-label, focus-visible, prefers-reduced-motion	Низкий
Интернационализация	i18n framework для английского/русского	Низкий

Таблица 4. Фаза 3 — Полировка и продакшн

9. Заключение

Проект SuperAgents OS демонстрирует амбициозную архитектуру с чётким разделением ответственности: ядро (EventBus + Kernel + DSL), сервисный слой (20+ singleton-сервисов), слой провайдеров (мульти-API адаптеры) и богатый UI (22+ панели). Основной функционал — оркестрация AI-агентов через DAG с событийно-ориентированной архитектурой — работает на уровне прототипа.

Однако текущее состояние проекта (~5/10) не позволяет рекомендовать его для продакшн-использования. Ключевые препятствия:

1. **16 критических дефектов** безопасности и корректности (инъекция кода в Sandbox, утечка API-ключей, деление на ноль, потеря данных в PluginSDK)
2. **Систематическое отсутствие runtime-валидации** данных на всех уровнях
3. **Фрагментированная персистенция** — 6 различных стратегий хранения без атомарности
4. **15+ нефункциональных UI-элементов** и 10+ панелей с фейковыми/захардкоженными данными
5. **Минимальное тестовое покрытие** — 5 тестовых файлов на ~80 исходных

Реализация трёхфазного плана (критические исправления → архитектура → полировка) позволит довести проект до уровня 9/10 за 8-12 недель. Фаза 1 (критические исправления) должна быть выполнена немедленно — без неё система уязвима и ненадёжна.